

# WebApp - Infraestrutura, Deploy e HTTPS

Mapa rápido da infraestrutura utilizada no projeto Flask

## 1. Visão geral

O WebApp está hospedado em uma VPS Debian e utiliza uma arquitetura em camadas. O Apache recebe as conexões externas, o Gunicorn executa a aplicação Flask, o systemd mantém o processo ativo, o PostgreSQL fornece persistência e o HTTPS é terminado no Apache por meio de certificado TLS.

```
INTERNET
|
| HTTPS :443
v
+-----+
| Apache |
| Reverse Proxy |
+-----+
|
| HTTP interno - 127.0.0.1:5153
v
+-----+
| Gunicorn |
| Servidor WSGI |
+-----+
|
|
v
+-----+
| Flask |
| WebApp |
+-----+
|
|
v
+-----+
| PostgreSQL |
+-----+
```

## 2. Componentes principais

| Componente   | Papel                                                           | No WebApp                                                   |
|--------------|-----------------------------------------------------------------|-------------------------------------------------------------|
| VPS / Debian | Sistema operacional e servidor onde os serviços são executados. | Base da infraestrutura.                                     |
| UFW          | Firewall do servidor.                                           | Controla portas externas como 22, 80 e 443.                 |
| Apache       | Servidor HTTP/HTTPS e reverse proxy.                            | Recebe webapp.jhan.com.br e encaminha para o Gunicorn.      |
| TLS / HTTPS  | Criptografa a comunicação entre navegador e servidor.           | Certificado configurado no Apache.                          |
| Gunicorn     | Servidor WSGI de produção.                                      | Executa o Flask em 127.0.0.1:5153.                          |
| Flask        | Framework da aplicação.                                         | Routes, autenticação, templates, regras e acesso aos dados. |
| systemd      | Gerenciador de serviços do Linux.                               | Mantém webapp.service ativo e reinicia em caso de falha.    |
| uv           | Gerencia ambiente e dependências Python.                        | Executa o Gunicorn no ambiente correto.                     |

| Componente | Papel                      | No WebApp                       |
|------------|----------------------------|---------------------------------|
| PostgreSQL | Banco de dados relacional. | Armazena os dados persistentes. |

### 3. Apache - porta de entrada

O Apache é a camada pública do WebApp. O navegador não acessa diretamente o Flask nem o Gunicorn. O domínio aponta para a VPS e o Apache recebe as requisições nas portas HTTP/HTTPS.

```
Navegador
|
| webapp.jhan.com.br
v
Apache :80 / :443
|
+--> Reverse Proxy --> 127.0.0.1:5153
```

Essa separação permite que o Apache cuide de domínio, HTTPS, redirecionamentos e proxy, enquanto a aplicação Python permanece isolada da Internet.

### 4. Gunicorn - servidor da aplicação Flask

O WebApp utiliza **Gunicorn**, e não Uvicorn. Flask é tradicionalmente WSGI; Uvicorn é um servidor ASGI, mais comum com frameworks como FastAPI.

No serviço atual, o Gunicorn é iniciado aproximadamente desta forma:

```
uv run gunicorn --bind 127.0.0.1:5153 app:app
```

**127.0.0.1** significa que a porta 5153 fica disponível somente dentro da VPS. Ela não precisa ser liberada publicamente no firewall.

### 5. systemd - mantendo o WebApp ativo

O systemd transforma o WebApp em um serviço Linux, permitindo inicialização automática e reinício em caso de falha.

```
systemd
|
+-- webapp.service
|
+-- uv run gunicorn
|
+-- Flask app:app
```

No projeto, o serviço usa o diretório **/home/hilaneto/webapp**, executa o Gunicorn em **127.0.0.1:5153** e utiliza **Restart=always**.

```
sudo systemctl status webapp
sudo systemctl restart webapp
sudo journalctl -u webapp -f
```

### 6. uv - ambiente Python

O uv gerencia o ambiente e as dependências Python. O systemd chama o Gunicorn por meio do uv, garantindo que a aplicação utilize as bibliotecas instaladas para o WebApp.

### 7. Flask - aplicação propriamente dita

Depois que o Gunicorn recebe a requisição interna, ele a entrega ao Flask. Entram então em ação app.py, Blueprints/routes, models, templates, static, autenticação e demais componentes.

```
Gunicorn
|
v
Flask app:app
|
+-- routes/
+-- models/
+-- database/
+-- templates/
+-- static/
```

## 8. PostgreSQL - persistência

O PostgreSQL fica na camada de dados. O navegador nunca conversa diretamente com o banco: o Flask realiza consultas e alterações e devolve ao usuário somente a resposta apropriada.

## 9. Firewall UFW

O UFW limita quais portas podem ser alcançadas externamente. Na infraestrutura do WebApp, as portas web 80 e 443 ficam disponíveis, além da porta 22 para administração SSH.

```
22/tcp -> SSH
80/tcp -> HTTP
443/tcp -> HTTPS

5153/tcp -> não precisa ser pública
Gunicorn escuta em 127.0.0.1
```

## 10. HTTPS e certificado TLS

O HTTPS é tratado na camada Apache. O navegador estabelece uma conexão criptografada com o Apache e, depois, o Apache encaminha internamente a requisição ao Gunicorn.

```
EXTERNO
Navegador ===== HTTPS/TLS =====> Apache :443

INTERNO
Apache ----- HTTP -----> Gunicorn :5153
|
v
Flask
```

O Flask não precisa administrar diretamente o certificado. A terminação TLS ocorre no Apache. O certificado pode ser emitido e renovado por uma autoridade como Let's Encrypt, normalmente com apoio do Certbot.

## 11. Caminho completo de uma requisição

1. Usuário acessa <https://webapp.jhan.com.br>
2. DNS resolve o domínio para o IP da VPS
3. UFW permite a conexão na porta 443
4. Apache recebe a conexão HTTPS
5. O certificado TLS protege a conexão
6. Apache atua como Reverse Proxy
7. Requisição segue para 127.0.0.1:5153
8. Gunicorn entrega a requisição ao Flask
9. Flask executa routes/models/database
10. PostgreSQL é consultado quando necessário
11. A resposta retorna pelo mesmo caminho

## 12. Por que essa arquitetura é interessante?

Cada componente possui uma responsabilidade definida. Apache cuida da exposição pública e HTTPS; Unicorn executa o Flask em produção; systemd controla o processo; UFW reduz a superfície exposta; e PostgreSQL permanece na camada de dados. Assim, o servidor de desenvolvimento do Flask não fica diretamente exposto à Internet.

## 13. Diagnóstico rápido

Quando o WebApp apresentar problema, é útil verificar a infraestrutura de fora para dentro:

1. DNS aponta corretamente para a VPS?
2. UFW permite 80/443?
3. Apache está ativo?
4. HTTPS/certificado está válido?
5. Reverse Proxy aponta para 127.0.0.1:5153?
6. webapp.service está ativo?
7. Unicorn está ouvindo na porta 5153?
8. Flask iniciou sem erros?
9. PostgreSQL está acessível?

Comandos úteis:

```
sudo ufw status verbose
sudo systemctl status apache2
sudo apachectl -S
sudo systemctl status webapp
sudo journalctl -u webapp -f
sudo ss -tulpn
```

## 14. Resumo mental

```
UFW = controla quem entra
Apache = recebe HTTP/HTTPS e faz o proxy
TLS = protege a comunicação externa
Unicorn = executa o Flask em produção
systemd = mantém o processo funcionando
uv = gerencia o ambiente Python
Flask = executa a aplicação WebApp
PostgreSQL = persiste os dados
```

## 15. Conclusão

A infraestrutura do WebApp é uma sequência de responsabilidades. A Internet chega ao firewall e ao Apache; o Apache termina o HTTPS e encaminha a requisição ao Unicorn; o Unicorn executa o Flask; e o Flask acessa o PostgreSQL quando necessário. systemd e uv dão suporte à execução contínua e ao ambiente Python. Esse mapa ajuda tanto a compreender o deploy quanto a diagnosticar falhas.